Universidad Mariano Gálvez de Guatemala

Carrera: Ingeniería Eléctrica

Jornada: Vespertina

Catedratico: Carlos Alberto Arias Lopez

Curso: Programación para la ciencia y la ingeniería 2

Proyecto #2

PPCYL2-AcadNet: Una solución web para la enseñanza personalizada

Massiel Ivanova Castellanos Pérez 1492-25-203

Rudy Rolando Mejia Marroquín 1492-25-1685

David Rolando Velásquez López 1492-25-197

Luis Rolando Veliz Vicente 1492-25-486

Guatemala, 10 de abril de 2025

Introducción

Actualmente el contexto de la educación digital que se tiene son las plataformas web que conectan a estudiantes con tutores. Estas se les solicita que no solo sean interfaces amigables, sino que también cuente con un respaldo técnico sólido que facilite procesar, almacenar y consultar información académica de manera eficiente. El presente proyecto, nombrado PPCYL2-AcadNet, surge como una respuesta a la necesidad planteada por la empresa PPCYL2-Games, la cual requiere una plataforma educativa que posibilite la gestión de usuarios, horarios, notas y reportes, todo ello a partir de archivos XML como suministro principal. El objetivo general del proyecto es desarrollar una solución que implemente una API bajo el protocolo HTTP utilizando el paradigma de programación orientada a objetos (POO). Como objetivos específicos se plantean: implementar una API en Python con el framework Flask, utilizar bases de datos para la persistencia de información de forma persistente, emplear archivos XML como medio de comunicación, aplicar expresiones regulares y estructuras de pila para la extracción de contenido textual, y construir una matriz dispersa manualmente bajo POO para representar el progreso temático de los estudiantes. El presente ensayo tiene como finalidad describir el diseño y la implementación de dicha solución, resaltando las decisiones técnicas aplicadas y su justificación.

Desarrollo

El proyecto PPCYL2-AcadNet se crea como una aplicación web bajo la arquitectura cliente-servidor. Según el enunciado "el frontend está desarrollado con Django, proporcionando interfaces limpias, seguras y administrables", mientras que "el backend está potenciado por Flask, responsable del procesamiento lógico y analítico de los datos, incluyendo servicios inteligentes y estructuras optimizadas como matrices dispersas" (Arias, 2025, p. 2). Django lo que promueve es un desarrollo rápido basado en el patrón MVT (Django Project, 2024). A la vez que Flask ofrece una arquitectura ligera para APIs (Flask Documentation, 2024). Esta separación permite avanzar cada componente independientemente y facilita que la API sea utilizada no solo por el frontend sino también por herramientas externas.

El backend comienza en app.py y almacena las rutas con el prefijo /api. La lógica central se encuentra en endpoints.py, en el cual se dirige el estado global del sistema entre ellos están los usuarios, horarios, notas y por último archivos cargados. Cuando el frontend se le manda un archivo XML, Flask lo verifica con la función _parse_xml() y lleva a cabo el parser correspondiente. Los datos obtenidos se guardan en el estado global STATE y se quedan en el archivo backend:state.json. El backend también fabrica una estructura llamada notas_indexsadas que autoriza ingresar rápidamente a las notas filtrando por carné y curso.

En el estudio del problema se identificaron requerimientos útiles para cada rol. El administrador, tiene acceso predeterminado con usuario. Este rol puede cargar archivos de entrada en XML y ver usuarios. Incluyendo, el sistema debe generar un XML de salida que debe contener el total de tutores y estudiantes cargados correctamente, así como asignaciones correctas e incorrectas. El administrador, después de iniciar sesión, ingresa a un panel central donde él puede cargar archivos XML, ver usuarios y consultar información general. También, cuenta con un apartado para cerrar sesión.

Los tutores cuentan con dos funciones principales. La primera es establecer horarios y la segunda es ingresar notas por medio de un XML, la información se guarda en una matriz dispersa creada por nosotros, todo eso se realiza por medio de POO. **Dond**e las filas son actividades y las columnas son carnets de estudiantes. El tutor utiliza el endpoint/api/tutor/notas para ingresar las notas. Dentro del frontend, el tutor cuenta de plantillas como tutor:horarios,html, tutor_notas.html, tutor_notas.html, tutor_reporte_promedio.html y tutor_top_notas.html. Asimismo algo que puede hacer el tutor son reportes gráficos utilizando Plotly, el reporte será exportado a PDF; además el reporte de la matriz dispersa será generado con Graphviz. Plotly permite gráficos interactivos exportables a PDF (Plotly, 2024), y Graphviz visualiza gráficos (Graphiz, 2024). Lo que contienen estos reportes son el promedio de notas por actividades y el TOP de notas de cierta actividad,

estructurado de mayor a menor. Cada vista del tutor se verifica el rol por medio de la sesión antes de mostrar cualquier información.

Los estudiantes, por otro lado, podrán revisar las notas si es que su tutor ya se las ha subido. La información estará visualizada en una tabla. Cuando un estudiante quiere buscar alguna nota, Django manda una petición a /api/estudiante/notas y Flask busca la información en la estructura de notas indexadas. El estudiante tiene un flujo de solo consulta, sin opción de cambiar cualquier dato, lo que asegura la honestidad de la información. Django lo que hace es controlar el ingreso según el rol y el backend lo que hace es solo enviar resultados ya procesados.

El frontend fue desarrollado en Django siguiendo una arquitectura modular con herencia de templates y separación de responsabilidades. El archivo manege.py es el punto de entrada del proyecto, y urls.py centraliza las rutas principales autorizadas a cada aplicación. La aplicación login incluye la lógica de autenticación, roles y navegación. El login se ejecuta mediante el endpoint/api/login donde Flask verifica las credenciales contra los usuarios almacenados y devuelve los datos del usuario.

Conclusión

El desarrollo de PPCLYL2-AcadNet permitió satisfacer todos los objetivos planteados. Logrando implementar un API funcional en Flask que ofrece servicios HTTP para la gestión académica. Lo que se utilizó fue la programación orientada a objetos tanto en la lógica de negocio como en como la implementación de la matriz dispersa. La cual representa de manera eficiente el progreso temático de los estudiantes con filas correspondientes a actividades y columnas a carnets. La separación de responsabilidades entre Django para el frontend y Flask para el backend demostró ser una estrategia efectiva. Lo que hizo Django fue encargarse de la presentación, navegación y control de sesiones en el frontend. Mientras Flask hizo el proceso de la lógica compleja, validó los archivos XML y gestionó el estado global del sistema mediante el archivo backend_state.json, permitiendo que la API pudiera ser consumida no solo por el frontend sino también por herramientas externas. Ahora para los requerimientos por rol, cada uno cumple con lo solicitado. Como limitación principal, la matriz dispersa en memoria podría perderse al reiniciar el servidor, aunque se resolvió mediante almacenamiento persistente en JSON. A partir de los resultados obtenidos de la creación del proyecto, PPCYL2-AcadNet cumple satisfactoriamente con los objetivos del proyecto y sienta una base sólida para futuros crecimientos en el ámbito de la educación digital personal.